Artificial Intelligence (AI) for Software

By Vasu Vuppala, Fall 2021

> **Note:** I originally wrote this article in late 2021 for Delta Dental of Michigan, where it was published internally, about a year before ChatGPT was released. It is republished here with permission. The text is unchanged except that internal system names have been replaced with generic descriptions and redundant duplicate lines were removed from one code sample. The code uses the OpenAI Python library as it existed in 2021 (`openai.Completion` with the `davinci` and `davinci-codex` engines), which has since been retired, and some of the original links may no longer work.

After many winters of setbacks, AI Spring is in full bloom mostly due to improvements in hardware, better algorithms especially in Deep Learning, and abundance of data. It is being used in image processing, language translation, chatbots, spam detection, product recommendation, speech recognition, text summarization, game playing, disease prediction, IT security, autonomous driving etc. It is being applied in every domain including dental healthcare (DL in Dentistry, Bone Loss Detection, startups like OverJet). Quite a lot of it is fascinating (MuZero, Assistants like Siri/Alexa, Live Portrait, AutoDraw, Google Translate, Tesla AutoPilot) and quite a bit is hype. Lately, Deep Learning (a sub-field of Machine Learning, which is a sub-field of AI) is being applied to the field of software development, so let us look at a few of these technologies.

GPT-3

Natural Language Processing (NLP) is a branch of AI that is used to build systems that can process (understand) text or speech. An NLP-based system can be used to predict the next word in a sentence (e.g. auto-completion), classify a text (e.g. sentiment analysis), summarize an article etc. A language model is a key component of an NLP-based system, and is used to predict words (from speech, or part of a sentence etc).

Generative Pre-trained Transformer version 3 (GPT-3) is a large generic language model based on artificial neural networks, developed by OpenAI. It was trained on about 550 GB of text from web pages, Wikipedia, and thousands of digital books. It is a generic model that can be used for various tasks: text generation, summarization, conversation, translation, and classification. Since GPT-3 is pre-trained on an extensive and diverse dataset, it can successfully perform a surprising number of NLP tasks without users providing any additional example data (i.e. training).

GPT-3 was released in mid-2020 and was found to be astonishing (it still is), and since then it has been used for a myriad of applications: writing blogs, stories, screenplays, poems, generating advice on the fly, summarizing complex documents, creating artistic ad copies etc. Unlike previous systems, it has empowered the general public, with little to no background in AI, to build AI-based systems. GPT-3 is proprietary and is available using APIs (Model-as-a-Service). Until recently these APIs were not available to everyone; one had to wait for a long time to get access, however, since last month, it has been made available to public at large (however it is not free).

There are other language models similar to but not as comprehensive as GPT-3: BERT (Google), XLNet (CMU/Google), GPT-Neo (open source).

OpenAI API: The Prompt

The OpenAI API is a RESTful interface to interact with the GPT-3 models. The client sends a *prompt* and the GPT-3 model responds to the prompt. For example, if we give it the prompt *"Delta Dental is a"*, it will respond with the following:

```
Delta Dental is a nonprofit organization and is the largest dental benefits provider in the United States.
```

The structure and the verbiage of the prompt influences the response from GPT-3. So designing the prompt is all the 'programming' that is needed. We can also choose the length, creativity, and randomness of the response, besides the execution engine (GPT-3 has several).

Here are some basic examples of GPT-3 usage written in Python. Note that the same can be achieved with simple curl commands.

Generation

The following code elicits the response for the prompt *"Delta Dental is a"*. The response will change based on the parameters (`max_tokens`, `temperature`, `top_p` etc). The response is a JSON object, which is parsed and displayed by the function `print_response()`, which is not included in the code.

GPT: Generation

```
import os
import openai

# API Key
openai.api_key = os.getenv("OPENAI_API_KEY")

# Set parameters to GPT
max_tokens = 32
temperature = 0.9
top_p = 1
frequency_penalty = 0
presence_penalty = 0
stop = ["---"]

prompt = """Delta Dental is a"""

response = openai.Completion.create(engine="davinci", prompt=prompt,
    temperature=temperature, top_p=top_p, max_tokens=max_tokens,
    frequency_penalty=frequency_penalty, presence_penalty=presence_penalty,
    stop=stop, echo=True)

print_response(response)
```

Sentiment Analysis

The prompt to gauge the sentiment in a text is as follows (check the last line):

```
The rep absolutely had no interest in helping or offering any viable solutions. I asked that they honor a crown that needs to be replaced because of a botched procedure in 2018. They wanted me to write a letter to their corporate office explaining my request and wait over 30 days for a response. When I asked why I can't just do this online, How does anyone still use snail mail? The rep had no explanation and no desire to help understand anything. She simply wanted to get me off the phone.

Sentiment (positive, neutral, negative):
```

And the response from GPT-3 is "Negative".

Translation

The following prompt tells GPT-3 to translate from English to Spanish:

```
English: Delta Dental is a nonprofit organization and is the largest dental benefits provider in the United States.

Spanish:
```

And the response from GPT-3 is:

```
Delta Dental es una organización sin fines de lucro y es la mayor proveedora de beneficios dentales en los Estados Unidos.
```

Check examples of GPT-3's generation, translation, classification, summarization, and conversation capabilities here.

Codex

GPT-3 is an interesting technology for NLP but how does it help with software development? Recall that GPT-3 was trained on a large amount of data from the web, which includes a lot of content in formal languages (think of GitHub) besides natural languages. Codex is a GPT-3 engine that specializes in formal languages. It can be used to generate programs from natural language specifications, or contrarily describe a program in natural language.

Examples of Codex

The following uses Codex to generate code for sorting a list of integers. The prompt is set to `/* A function to sort a list of integers in Java */`. Note that it is in the format of a Java comment, which signals Codex to generate the code in Java (the "in Java" phrase is needed, otherwise Codex will generate code in C because C also uses the same syntax).

Codex: Java

```
import os
import openai

# API Key
openai.api_key = os.getenv("OPENAI_API_KEY")

# Set parameters to GPT
max_tokens = 256
temperature = 0.1
top_p = 1
frequency_penalty = 0
presence_penalty = 0

prompt = """/* A function to sort a list of integers in Java */"""

response = openai.Completion.create(engine="davinci-codex", prompt=prompt,
    temperature=temperature, top_p=top_p, max_tokens=max_tokens,
    frequency_penalty=frequency_penalty, presence_penalty=presence_penalty,
    echo=False)

print_response(response)
```

Here is the response from Codex. As you can see, the result is accurate and uses SelectionSort.

Codex: Java Response

```java
public class Sort {
    public static void main(String[] args) {
        int[] list = {5, 4, 3, 2, 1};
        sort(list);
        for (int i = 0; i < list.length; i++) {
            System.out.println(list[i]);
        }
    }

    public static void sort(int[] list) {
        for (int i = 0; i < list.length; i++) {
            for (int j = i + 1; j < list.length; j++) {
                if (list[i] > list[j]) {
                    int temp = list[i];
                    list[i] = list[j];
                    list[j] = temp;
                }
            }
        }
    }
}
```

Changing the prompt to `/* A function to sort a list of integers in Java using quicksort */` will generate a code snippet that uses the QuickSort algorithm. Codex can be used to generate and explain code in many programming languages including SQL and HTML. Interestingly, it can also generate the backend and frontend codes for a web-based application. Watch this video, and try GitHub Copilot to appreciate Codex's capabilities.

Codex is not open to the public yet; one has to apply to get access (it is not clear what the criteria is; for me, it took 4-5 days of waiting).

CodeNet

IBM's CodeNet is a large curated dataset of code samples. It has about 14 million annotated code samples with 500 million lines of code from 50+ different programming languages. It is useful for developing AI models that generate code. It is akin to ImageNet for images, and can be used to develop your own models (like Codex). IBM claims to have used its AI for Code technologies to refactor a large (1 million lines of code) monolithic Java application into micro-services in about four weeks (see https://research.ibm.com/blog/codenet-ai-for-code). CodeNet is public domain and can be accessed at https://github.com/IBM/Project_CodeNet. CodeNet will be useful to develop and/or evaluate our own AI models.

Applications

GPT-3 has been used to write applications that create images based on text captions (DALL·E) or rap like Eminem but is it really useful for us? Here are some use cases where natural and formal language based technologies will or will not be useful to us (these are my opinions).

Application	Comment
Making developers more efficient and write better code	Yes
Find bugs in programs	Yes
Predicting and/or diagnosing production incidents	No. For production incidents, we will need to develop a specialized AI model.
Migrate a legacy application to our cloud platform more efficiently	Yes (based on IBM's claim)
Translating business rules from/to English	Yes (by providing additional training data to GPT-3)
Provide natural language interface to enterprise applications	Yes
Fraud detection	No. Will need to develop a specialized AI model.
Predict claim payments	No. Claim payments are deterministic (rule based).
Analyze X-Rays	No. Will require developing an image processing model.

Conclusion

AI for software development appears promising but is not going to replace software engineers anytime soon. However, it can help them become better and more productive. It can also help in modernizing applications written with older technology stacks. AI, in general, can be used for various applications at Delta but with caution as these technologies are nascent, and many will not live up to their promises (Dr Andrew Ng gives prudent advice on AI rollout in an enterprise: What AI Can or Cannot Do, How to Choose First AI Project).

© 2021 Vasu Vuppala. Text licensed under CC BY 4.0. Code samples licensed under the MIT License.